Corrections — Examen Python ING-3

Session Principale 2024-2025 • TEK-UP

Exercice 1 (5 pts) — Lecture de sorties de code

1. Code 1 (1 pt)

```
file = open('abc.txt')
a = [i for i in file]
print(a)
```

➜ Sortie : ['abc\\n', 'def\\n', 'ghi']

open() sans mode ni strip() conserve les sauts de ligne \n. La compréhension de liste place chaque ligne (avec son \n) dans la liste. La dernière ligne n'a pas de \n si le fichier ne se termine pas par une ligne vide.

2. Code 2 (1 pt)

```
a = "ab cd"
print('*'.join(a.split()))
```

➜ Sortie : ab*cd

a.split() découpe sur les espaces → ['ab', 'cd']. ''.join(...) les réunit avec '*' → 'ab*cd'.*

3. Code 3 (0.5 pt)

```
a = "abcdef"
a[1] = 'x'
print(a)
```

➜ Erreur : TypeError — 'str' object does not support item assignment

Les chaînes de caractères sont immuables en Python. L'affectation a[1]='x' lève une TypeError et le programme s'arrête.

4. Code 4 (0.5 pt)

```
mois = ['Mai', 'Juin', 'Juillet']
print(mois[-2][-2])
```

➜ Sortie : i

mois[-2] → 'Juin' | 'Juin'[-2] → 'i' (J=0, u=1, i=2, n=3 ; -2 = index 2 = 'i')

5. Code 5 (1 pt)

```
a = [1,2,3,4,5,6,7,8,9]
b = [i if i%3==0 else 0 for i in a]
print(b)
```

➜ Sortie : [0, 0, 3, 0, 0, 6, 0, 0, 9]

Pour chaque i : si i divisible par 3 → i, sinon → 0. Multiples de 3 dans [1..9] : 3, 6, 9.

6. Code 6 (1 pt)

```
print((lambda x: x[::-1].upper())("bonjour"))
```

➜ Sortie : RUOJNOB

"bonjour"[::-1] inverse la chaîne → "ruojnob", puis .upper() met en majuscules → "RUOJNOB".

Exercice 2 (5 pts) — Gestion des salariés

Structure d'un salarié : dictionnaire avec clés **nom**, **email**, **projets** (liste de tuples (nom_projet, heures_travaillées, taux_horaire)).

1. ajouter_salarie() (1.5 pt)

```
BD_salaries = []   # liste globale

def ajouter_salarie():
    nom = input('Nom : ').strip()
    while not nom:
        print('Le nom ne peut pas être vide.')
        nom = input('Nom : ').strip()

    email = input('Email : ').strip()
    while '@' not in email or '.' not in email:
        print('Email invalide.')
        email = input('Email : ').strip()

    projets = []
    nb = int(input('Nombre de projets : '))
    for _ in range(nb):
        nom_p = input('  Nom du projet : ').strip()
        heures = float(input('  Heures travaillées : '))
        taux   = float(input('  Taux horaire (€/h) : '))
        projets.append((nom_p, heures, taux))

    salarie = {'nom': nom, 'email': email, 'projets': projets}
    BD_salaries.append(salarie)
    print(f'Salarié {nom} ajouté avec succès.')
```

Contrôles : nom non vide, email contenant '@' et '.', heures/taux convertis en float.

2. salaire_total() (1.5 pt)

```
def salaire_total(email):
    for salarie in BD_salaries:
        if salarie['email'] == email:
            total = sum(h * t for _, h, t in salarie['projets'])
            return total
    print('Salarié introuvable.')
    return None
```

Pour chaque projet, on multiplie heures_travaillées × taux_horaire et on somme le tout.

3. classement_salaries() (2 pt)

```
def classement_salaries(nom_projet):
    def heures_sur_projet(salarie):
        for nom_p, heures, _ in salarie['projets']:
            if nom_p == nom_projet:
                return heures
        return 0   # salarié non affecté à ce projet
    classement = sorted(BD_salaries,
                        key=heures_sur_projet,
                        reverse=True)
    print(f'Classement pour le projet "{nom_projet}" :')
    for i, s in enumerate(classement, 1):
        h = heures_sur_projet(s)
        print(f'  {i}. {s["nom"]} — {h}h')
    return classement
```

La fonction clé recherche les heures du projet donné pour chaque salarié. Si le salarié n'est pas affecté au projet, on retourne 0 (il arrive en dernier).

Exercice 3 (10 pts) — Gestion d'un Parking (POO)

1a. Classe Voiture — constructeur (0.75 pt)

```
class Voiture:
    def __init__(self):
        self.immatriculation = input('Immatriculation : ').strip()
        self.est_garee = False   # non garée par défaut
```

1b. Méthode afficher() (0.75 pt)

```
    def afficher(self):
        etat = 'Garée' if self.est_garee else 'Non garée'
        print(f'Immatriculation : {self.immatriculation}')
        print(f'État            : {etat}')
```

2a. Classe Parking — constructeur paramétré (1 pt)

```
class Parking:
    def __init__(self, nom, capacite):
        self.nom      = nom
        self.capacite = capacite
        self.__voitures = []   # liste privée, initialement vide
```

__voitures (double underscore) rend l'attribut privé par name mangling en Python.

2b. afficher_parking() (1 pt)

```
    def afficher_parking(self):
```

```
        print(f'Parking  : {self.nom}')
        print(f'Capacité : {self.capacite} places')
        print(f'Occupées : {len(self.__voitures)} / {self.capacite}')
```

2c. afficher_voitures() (1 pt)

```
    def afficher_voitures(self):
        if not self.__voitures:
            print('Aucune voiture garée.')
            return
        print(f'--- Voitures dans {self.nom} ---')
        for v in self.__voitures:
            v.afficher()
            print()
```

2d. ajouter_voiture(immatriculation) (1.25 pt)

```
    def ajouter_voiture(self, immatriculation):
        if len(self.__voitures) >= self.capacite:
            print('Parking Plein')
            return
        # Créer la voiture avec l'immatriculation donnée
        v = object.__new__(Voiture)
        v.immatriculation = immatriculation
        v.est_garee = True
        self.__voitures.append(v)
        print(f'Voiture {immatriculation} ajoutée au parking {self.nom}.')
```

On utilise object.__new__(Voiture) pour créer l'objet sans déclencher input(). L'immatriculation est passée directement par le programme principal.

2e. sortir_voiture(immatriculation) (1.25 pt)

```
    def sortir_voiture(self, immatriculation):
        for v in self.__voitures:
            if v.immatriculation == immatriculation:
                v.est_garee = False
                self.__voitures.remove(v)
                print(f'Voiture {immatriculation} a quitté le parking.')
                return
        print('Voiture introuvable')
```

3. Programme principal (3 pts)

3a. Demander le nombre de parkings (0.25 pt)

```
nb = int(input('Combien de parkings souhaitez-vous gérer ? '))
```

3b. Instancier et stocker dans un dictionnaire (1.5 pt)

```
dict_parkings = {}   # dict[str, Parking]
for _ in range(nb):
    nom_p = input('Nom du parking : ').strip()
    cap   = int(input('Capacité : '))
    dict_parkings[nom_p] = Parking(nom_p, cap)
```

3c. Saisir parking + immatriculation, ajouter et afficher (1.25 pt)

```
nom_parking = input('Nom du parking : ').strip()
immat       = input('Immatriculation de la voiture : ').strip()

if nom_parking in dict_parkings:
    dict_parkings[nom_parking].ajouter_voiture(immat)
    dict_parkings[nom_parking].afficher_voitures()
else:
    print("Ce parking n'existe pas.")
```

Code complet — Exercice 3

```
class Voiture:
    def __init__(self):
        self.immatriculation = input('Immatriculation : ').strip()
        self.est_garee = False

    def afficher(self):
        etat = 'Garée' if self.est_garee else 'Non garée'
        print(f'Immatriculation : {self.immatriculation}')
        print(f'État            : {etat}')

class Parking:
    def __init__(self, nom, capacite):
        self.nom = nom
        self.capacite = capacite
        self.__voitures = []

    def afficher_parking(self):
        print(f'Parking  : {self.nom}')
        print(f'Capacité : {self.capacite} places')
        print(f'Occupées : {len(self.__voitures)} / {self.capacite}')

    def afficher_voitures(self):
        if not self.__voitures:
            print('Aucune voiture garée.')
            return
        for v in self.__voitures:
            v.afficher()
```

```
            print()

    def ajouter_voiture(self, immatriculation):
        if len(self.__voitures) >= self.capacite:
            print('Parking Plein')
            return
        v = object.__new__(Voiture)
        v.immatriculation = immatriculation
        v.est_garee = True
        self.__voitures.append(v)
        print(f'Voiture {immatriculation} ajoutée.')

    def sortir_voiture(self, immatriculation):
        for v in self.__voitures:
            if v.immatriculation == immatriculation:
                v.est_garee = False
                self.__voitures.remove(v)
                print(f'{immatriculation} a quitté le parking.')
                return
        print('Voiture introuvable')

# ■■ Programme principal ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
nb = int(input('Combien de parkings ? '))
dict_parkings = {}
for _ in range(nb):
    nom_p = input('Nom du parking : ').strip()
    cap   = int(input('Capacité : '))
    dict_parkings[nom_p] = Parking(nom_p, cap)

nom_parking = input('Nom du parking : ').strip()
immat       = input('Immatriculation : ').strip()
if nom_parking in dict_parkings:
    dict_parkings[nom_parking].ajouter_voiture(immat)
    dict_parkings[nom_parking].afficher_voitures()
else:
    print("Ce parking n'existe pas.")
```